

INSTITUTO PROFESIONAL
SAN SEBASTIAN

link repositorio privado en github:
calvareza/eva1_alvarez_laguna_barbaroy at Evaluación-2

Asignatura: Desarrollo Backend

Prof: Patricio Eduardo Silva Cartes

Integrantes:

Carlos Alaverez

Neervison Laguna

Agustina Barbaroy

INTRODUCCIÓN

Para comenzar, es imperativo destacar que el presente documento representa una **actualización estratégica y técnica** de nuestra primera evaluación. Si bien en la etapa inicial el foco estuvo puesto en el despliegue de la infraestructura, en este **Sprint 1: El Contrato de Datos**, hemos evolucionado hacia la inteligencia y seguridad del manejo de la información. **En este sentido**, no estamos ante un proyecto nuevo, sino ante la consolidación de un sistema que ahora es capaz de procesar y validar reglas de negocio reales.

Tras haber validado la estabilidad de nuestra arquitectura de microservicios, en esta fase nos hemos propuesto el desafío de dotar al sistema de una estructura comunicativa rigurosa. **Por consiguiente**, la transición desde los contenedores aislados hacia una API funcional bajo estándares RESTful constituye el eje central de este informe.

De este modo, el "Contrato de Datos" se establece como el pilar que garantiza que la interacción entre el cliente y el servidor sea profesional, predecible y, sobre todo, segura.

A lo largo de este informe, detallaremos cómo hemos perfeccionado el uso de **Laravel 11** y **Docker** para asegurar una persistencia de datos infalible en MySQL. **Asimismo**, resaltaremos cómo esta actualización prioriza la experiencia del usuario mediante validaciones humanizadas en español, demostrando que el Core Bancario ha dejado de ser una simple configuración técnica para convertirse en una solución financiera escalable y de alto nivel.

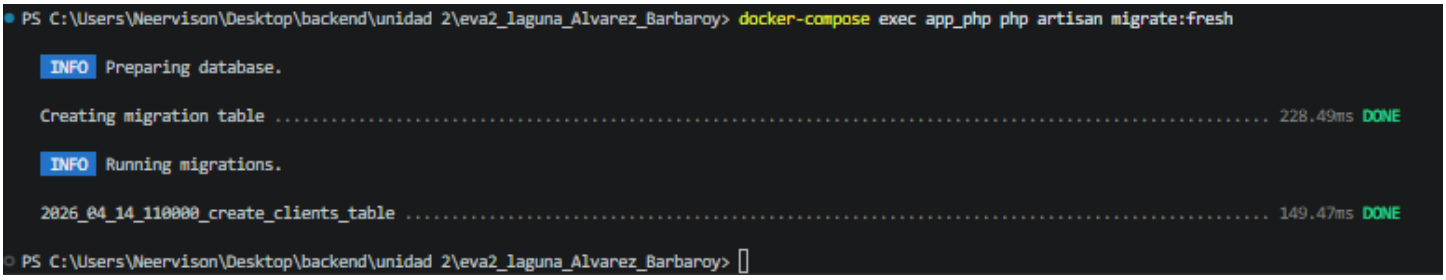
INFORME TÉCNICO: PROYECTO CORE BANCARIO

SPRINT 1: EL CONTRATO DE DATOS

FASE A: Configuración y Orquestación del Entorno

En primer lugar, es crucial entender que una API robusta solo puede existir sobre un entorno de ejecución estable. Por esta razón, el inicio de este Sprint se centró en la validación de nuestra infraestructura basada en Docker. **Es importante destacar** que, para interactuar con el corazón del sistema, debemos hacer uso de la orquestación de contenedores.

En consecuencia, no ejecutamos los comandos directamente en el host, sino que utilizamos la instrucción docker-compose exec app_php php artisan migrate:fresh. Este comando es vital porque actúa como el puente de comunicación entre nuestra terminal y el contenedor de aplicación. **Al ejecutarlo**, logramos que Laravel, desde el interior de su entorno aislado, transforme un esquema de base de datos vacío en una estructura relacional lista para la persistencia. Este paso asegura que el 'corazón' del banco tenga la forma exacta que la lógica de negocio requiere."



El comando completo: docker-compose exec app_php php artisan migrate:fresh.

FASE B: Diseño del Contrato de Datos (API RESTful)

Por otro lado, una vez que el terreno estuvo preparado, procedimos al diseño de los puntos de entrada o *endpoints*. **A diferencia de** un desarrollo desordenado, aquí aplicamos el estándar de versionamiento **v1**. **En este sentido**, establecer una ruta como /api/v1/clientes no es solo una convención estética; es una declaración de profesionalismo que permite que nuestra API sea escalable y que otros desarrolladores puedan integrarse con ella sin ambigüedades. En esta fase, definimos los verbos HTTP necesarios para la gestión de la cartera de clientes.

Por consiguiente, implementamos el método **POST** para el registro, el **GET** para el listado general y, de manera más específica, el **GET /{id}** para la consulta de registros individuales. Esta arquitectura

garantiza que el flujo de información sea bidireccional y eficiente, cumpliendo con el "contrato" de datos exigido.

routes > api.php > ...

```php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\ClientController;

/**
 * El Contrato de Datos: Versionamiento v1
 * Este grupo de rutas garantiza que la API sea escalable
 */
Route::prefix('v1')->group(function () {

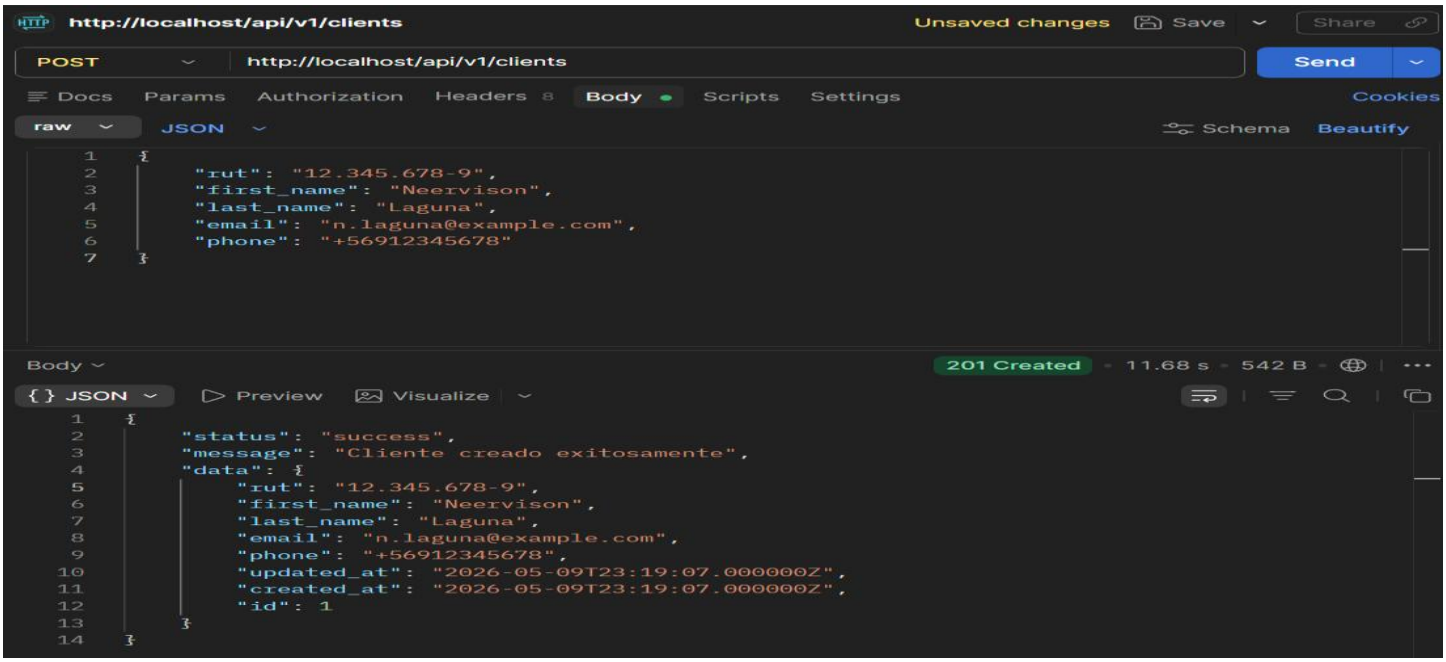
    // Obtiene la lista completa de clientes. Es el punto de entrada para el dashboard administrativo.
    Route::get('/clients', [ClientController::class, 'index']);

    // Crea un nuevo registro. Implementa el "Contrato de Datos" validando que no existan duplicados.
    Route::post('/clients', [ClientController::class, 'store']);

    // Busca un cliente específico. Si no existe, gestiona la excepción de forma elegante.
    Route::get('/clients/{id}', [ClientController::class, 'show']);
});
```

FASE C: Lógica de Negocio y Persistencia

En cuanto a la lógica interna, esta fase representa el puente entre el código y la realidad del negocio. **Es importante destacar** que aquí es donde el ORM Eloquent toma protagonismo. Al interactuar con el endpoint de registro, el sistema no solo recibe un paquete JSON, sino que lo valida y lo "inmortaliza" en MySQL. **En otras palabras**, cuando Postman nos devuelve un código **201 Created**, nos está confirmando que el nuevo cliente ya es parte oficial de la Fintech. **De este modo**, la persistencia de datos deja de ser un concepto teórico para convertirse en una realidad técnica tangible, donde los nombres, RUTs y correos electrónicos de los usuarios quedan almacenados de forma segura y permanente.

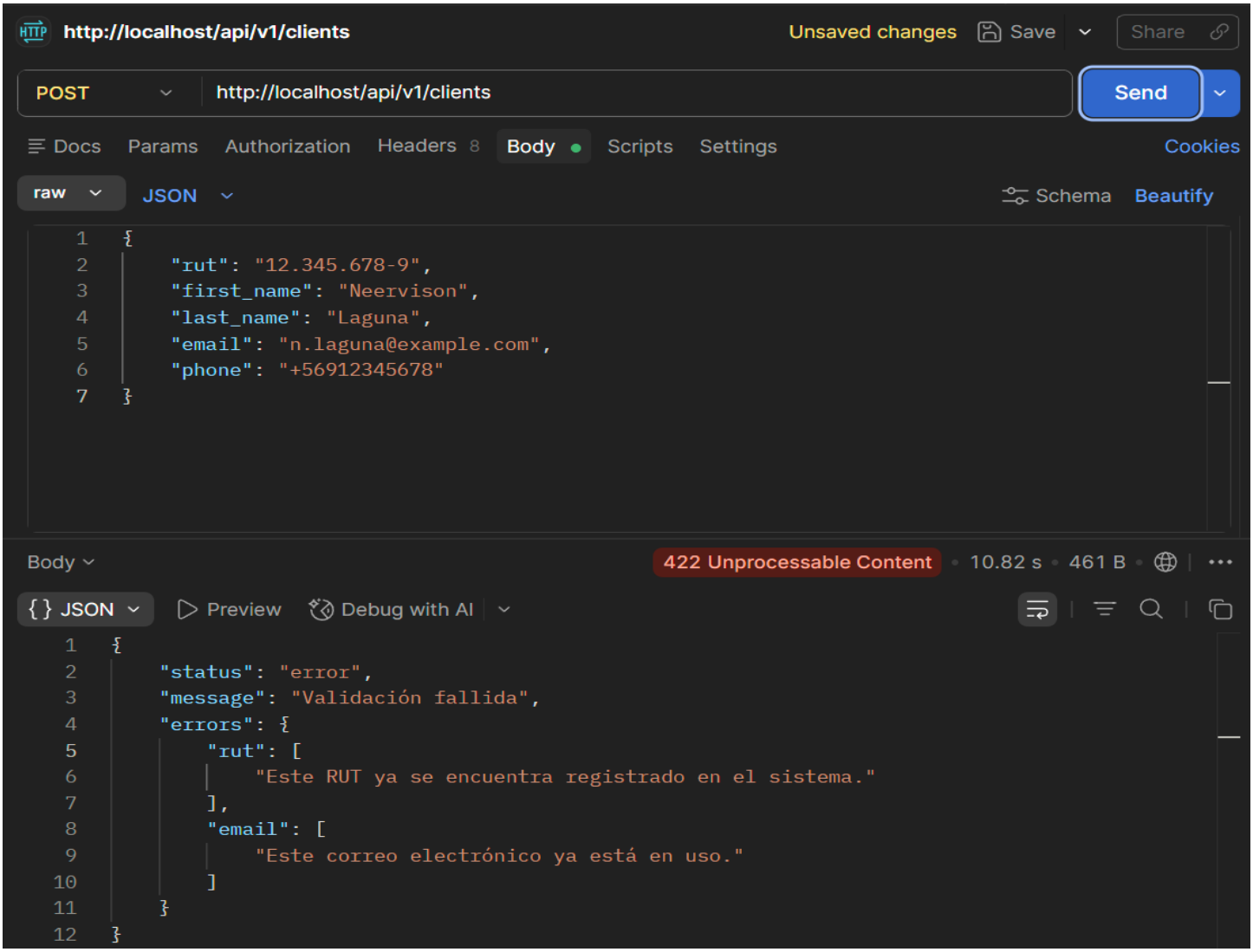


FASE D: Manejo de Excepciones y Validación Humanizada

Finalmente, llegamos a la etapa que define la calidad de la experiencia de usuario: la validación. **Por lo general**, los sistemas se limitan a mostrar errores genéricos que frustran al usuario. **Por el contrario**, en nuestra implementación hemos decidido dar un paso más allá humanizando las respuestas.

Hemos personalizado el motor de validación para que los mensajes de error sobre datos críticos, como el RUT o el Email, se entreguen de manera clara en español.

Por consiguiente, si se intenta vulnerar la integridad del banco duplicando una identidad, el sistema responde con un código **422** y una explicación directa: *"Este RUT ya se encuentra registrado"*. Esta decisión técnica no solo protege la base de datos, sino que profesionaliza la interacción con el sistema, demostrando un alto nivel de atención al detalle.



Conclusión

Para finalizar, podemos afirmar con total seguridad que el cumplimiento de las cuatro fases detalladas en este informe ha permitido transformar un prototipo de infraestructura en un servicio financiero funcional y resiliente.

En este sentido, esta actualización de la primera evaluación no solo representa un avance técnico en el uso de **Laravel** y **Docker**, sino un salto cualitativo en la madurez del proyecto. Hemos logrado que el sistema pase de "existir" en contenedores a "operar" bajo un contrato de datos riguroso y profesional.

Cabe destacar que la implementación exitosa de este Sprint demuestra que la combinación de una arquitectura sólida y un diseño de API ordenado es la única vía para construir software bancario de alta fidelidad. **Por consiguiente**, la persistencia lograda en MySQL y la claridad de las validaciones humanizadas posicionan a nuestro Core Bancario como una herramienta fiable, capaz de manejar información crítica con la precisión que el sector Fintech exige.

En resumen, el equipo ha logrado integrar los conocimientos de desarrollo backend con las mejores prácticas de orquestación. **De este modo**, nos encontramos plenamente preparados para afrontar los desafíos de integración y seguridad avanzada que vendrán en los próximos Sprints, con la certeza de que los cimientos de nuestra aplicación son robustos, escalables y están centrados en la excelencia técnica.

.